

Exam Prep – C Funktionen

Contents

1	Bitoperationen	4
1.1	set_bit	4
1.2	clear_bit	4
1.3	toggle_bit	4
1.4	get_bit	4
1.5	count_ones (Variante 1: Shift)	4
1.6	count_ones (Variante 2: Kernighan)	4
1.7	pack_rgb	5
1.8	unpack_rgb	5
1.9	bit_set	5
1.10	bit_clear	5
1.11	bit_toggle	5
1.12	bit_get	5
1.13	is_power_of_two	6
1.14	reverse_byte	6
2	Strings	7
2.1	is_palindrome	7
2.2	count_words	7
2.3	str_reverse	7
2.4	str_replace_char	7
2.5	str_trim	8
2.6	str_to_int	8
2.7	str_starts_with	8
2.8	str_count_vowels	9
2.9	str_to_upper	9
2.10	str_repeat	9
3	Encoding / Verschlüsselung	10
3.1	caesar_encode	10
3.2	caesar_decode	10
3.3	xor_encode	10
3.4	is_rotation	10
3.5	rle_encode	11
3.6	rle_decode	11
3.7	compress_rle	12
3.8	encrypt (Caesar auf Datei)	12
3.9	decrypt (Caesar aus Datei)	12
3.10	encode (RLE auf Datei)	13
3.11	decode (RLE aus Datei)	13
4	Rekursion	14
4.1	fast_power	14
4.2	gcd	14
4.3	hanoi	14
4.4	count_paths	14
4.5	count_paths_memo	14
4.6	fib (rekursiv)	15
4.7	fibonacci (iterativ-rekursiv)	15
4.8	digit_sum	15
4.9	binary_search (rekursiv)	15
4.10	count_char	15
4.11	calculate_arrays	16
5	Structs / Strukturen	17
5.1	typedef	17
5.2	student_average	17

5.3	find_best_student	17
5.4	sort_students_by_average	17
5.5	student_passed_all	18
5.6	typedef	18
5.7	account_deposit	18
5.8	account_withdraw	18
5.9	find_richest	18
5.10	total_balance_by_type	19
5.11	account_transfer	19
5.12	typedef	19
5.13	get_busiest_server	19
5.14	get_servers_in_location	20
5.15	filter_servers_by_utilization	20
5.16	typedef	20
5.17	find_event_by_id	20
5.18	find_most_expensive_event	20
5.19	filter_events_by_type	21
5.20	typedef	21
5.21	sort_notes	21
6	Warehouse / Inventory	22
6.1	typedef	22
6.2	warehouse_create	22
6.3	warehouse_destroy	22
6.4	warehouse_add_product	22
6.5	warehouse_remove_by_id	23
6.6	warehouse_find_by_name	23
6.7	warehouse_total_value	23
6.8	warehouse_sort_by_price	23
6.9	typedef	24
6.10	find_lowest_stock	24
6.11	apply_discount	24
7	Matrix	25
7.1	matrix_create / create_matrix	25
7.2	matrix_free / free_matrix	25
7.3	matrix_scale	25
7.4	matrix_add	25
7.5	matrix_is_symmetric	25
7.6	matrix_trace	26
7.7	transpose_matrix	26
8	Statistik	27
8.1	min_cycles	27
8.2	max_cycles	27
8.3	average_cycles	27
8.4	standard_deviation	27
8.5	convert_to_time	27
9	Profiler	29
9.1	typedef	29
9.2	create_profiler	29
9.3	add_function_call	29
9.4	sort_entries	29
9.5	destroy_profiler	30
10	Einfach verkettete Liste (Singly Linked List)	31
10.1	typedef	31
10.2	slist_create	31
10.3	slist_destroy	31
10.4	slist_push_front	32

10.5	slist_push_back	32
10.6	slist_pop_front	32
10.7	slist_remove_at	32
10.8	slist_insertion_sort	33
10.9	slist_reverse	33
10.10	slist_to_array	34
10.11	list_append (einfache Variante)	34
10.12	list_free	34
10.13	list_sum	34
10.14	list_remove_duplicates	35
10.15	list_is_sorted	35
10.16	list_insert_sorted	35
10.17	list_filter_even	36
11	Doppelt verkettete Liste (Doubly Linked List)	37
11.1	typedef	37
11.2	dlist_create	37
11.3	dlist_destroy	37
11.4	dlist_push_front	37
11.5	dlist_push_back	38
11.6	dlist_pop_back	38
11.7	dlist_is_consistent	38
12	Stack	40
12.1	typedef	40
12.2	stack_create	40
12.3	stack_destroy	40
12.4	stack_push	40
12.5	stack_pop	40
12.6	stack_peek	41
12.7	stack_is_empty	41
13	Queue (Warteschlange)	42
13.1	typedef	42
13.2	queue_create	42
13.3	queue_destroy	42
13.4	queue_enqueue	42
13.5	queue_dequeue	43
13.6	queue_peek	43
13.7	queue_is_empty	43
13.8	queue_size	43
14	Dynamisches Array	44
14.1	typedef	44
14.2	da_create	44
14.3	da_destroy	44
14.4	da_push	44
14.5	da_get	44
14.6	da_remove_at	45
14.7	da_min	45
14.8	da_max	45
14.9	da_mean	45
14.10	da_sort	46
15	File I/O	47
15.1	count_lines	47
15.2	read_ints_from_file	47
15.3	file_max	47
15.4	write_squares	48
15.5	number_lines	48

1 Bitoperationen

1.1 set_bit

```
uint32_t set_bit(uint32_t val, int pos)
{
    return val | (1U << pos);
}
```

Setzt Bit an Position pos in val durch OR-Verknüpfung mit einer Maske.

1.2 clear_bit

```
uint32_t clear_bit(uint32_t val, int pos)
{
    return val & ~(1U << pos);
}
```

Löscht Bit an Position pos mit AND NOT einer Maske. Das Tilde-Operator invertiert alle Bits der Maske.

1.3 toggle_bit

```
uint32_t toggle_bit(uint32_t val, int pos)
{
    return val ^ (1U << pos);
}
```

Kippt Bit an Position pos durch XOR: 0 wird 1, 1 wird 0.

1.4 get_bit

```
int get_bit(uint32_t val, int pos)
{
    return (val >> pos) & 1U;
}
```

Liest das Bit an Position pos aus: erst nach rechts schieben, dann mit 1 maskieren.

1.5 count_ones (Variante 1: Shift)

```
int count_ones(uint32_t val)
{
    int count = 0;
    while (val > 0) {
        count += val & 1;
        val >>= 1;
    }
    return count;
}
```

Zählt gesetzte Bits (Hamming-Gewicht) durch iteratives Prüfen des LSB und Rechtsschieben.

1.6 count_ones (Variante 2: Kernighan)

```
int count_ones(unsigned int n)
{
    int count = 0;
    while (n != 0) {
        n &= (n - 1);
        count++;
    }
    return count;
}
```

Kernighans Methode: $n \& (n-1)$ löscht das niedrigste gesetzte Bit. Läuft nur so oft wie Bits gesetzt sind, daher schneller als Variante 1.

1.7 pack_rgb

```
uint32_t pack_rgb(uint8_t r, uint8_t g, uint8_t b)
{
    return ((uint32_t)r << 16) | ((uint32_t)g << 8) | (uint32_t)b;
}
```

Packt drei 8-Bit-Farbkanäle in ein 32-Bit-Wort im Format 0x00RRGGBB.

1.8 unpack_rgb

```
void unpack_rgb(uint32_t packed, uint8_t *r, uint8_t *g, uint8_t *b)
{
    if (r != NULL) *r = (packed >> 16) & 0xFF;
    if (g != NULL) *g = (packed >> 8) & 0xFF;
    if (b != NULL) *b = packed & 0xFF;
}
```

Extrahiert die drei Farbkanäle aus einem gepackten 32-Bit-Wert durch Schieben und Maskieren mit 0xFF.

1.9 bit_set

```
unsigned int bit_set(unsigned int n, int pos)
{
    if (pos < 0 || pos > 31) return n;
    return n | (1 << pos);
}
```

Setzt Bit an Position pos mit Bereichsprüfung; gibt n unverändert zurück falls pos ungültig.

1.10 bit_clear

```
unsigned int bit_clear(unsigned int n, int pos)
{
    if (pos < 0 || pos > 31) return n;
    return n & ~(1 << pos);
}
```

Löscht Bit an Position pos mit Bereichsprüfung.

1.11 bit_toggle

```
unsigned int bit_toggle(unsigned int n, int pos)
{
    if (pos < 0 || pos > 31) return n;
    return n ^ (1 << pos);
}
```

Kippt Bit an Position pos mit Bereichsprüfung.

1.12 bit_get

```
int bit_get(unsigned int n, int pos)
{
    if (pos < 0 || pos > 31) return 0;
    return (n >> pos) & 1;
}
```

Liest Bit an Position pos mit Bereichsprüfung; gibt 0 zurück bei ungültigem Index.

1.13 is_power_of_two

```
int is_power_of_two(unsigned int n)
{
    if (n == 0) return 0;
    return (n & (n - 1)) == 0;
}
```

Prüft ob n eine Zweierpotenz ist. Zweierpotenzen haben genau ein gesetztes Bit, daher gilt $n \& (n-1) == 0$.

1.14 reverse_byte

```
unsigned int reverse_byte(unsigned int n)
{
    unsigned int result = 0;
    n &= 0xFF;
    for (int i = 0; i < 8; i++) {
        result <<= 1;
        result |= (n & 1);
        n >>= 1;
    }
    return result;
}
```

Kehrt die Bitreihenfolge eines einzelnen Bytes um. Die Schleife liest Bits von rechts und schreibt sie nach links in `result`.

2 Strings

2.1 is_palindrome

```
int is_palindrome(const char *s)
{
    if (s == NULL) return 0;
    int len = strlen(s);
    if (len == 0) return 1;
    int left = 0;
    int right = len - 1;
    while (left < right) {
        if (tolower(s[left]) != tolower(s[right])) return 0;
        left++;
        right--;
    }
    return 1;
}
```

Prüft ob ein String ein Palindrom ist (Groß-/Kleinschreibung ignoriert). Zwei-Zeiger-Ansatz von außen nach innen, $O(n)$.

2.2 count_words

```
int count_words(const char *s)
{
    if (s == NULL) return 0;
    int count = 0;
    int in_word = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] != ' ') {
            if (in_word == 0) { count++; in_word = 1; }
        } else {
            in_word = 0;
        }
    }
    return count;
}
```

Zählt Wörter (durch Leerzeichen getrennt) mit einem Zustandsflag `in_word`. Mehrfache Leerzeichen werden korrekt behandelt.

2.3 str_reverse

```
char *str_reverse(const char *s)
{
    if (s == NULL) return NULL;
    int len = strlen(s);
    char *reversed = malloc(len + 1);
    if (reversed == NULL) return NULL;
    for (int i = 0; i < len; i++)
        reversed[i] = s[len - 1 - i];
    reversed[len] = '\0';
    return reversed;
}
```

Gibt eine neu allokierte umgekehrte Kopie des Strings zurück. Der Aufrufer ist verantwortlich für `free()`.

2.4 str_replace_char

```
int str_replace_char(char *s, char old_char, char new_char)
{
    if (s == NULL) return -1;
    int count = 0;
```

```

    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == old_char) { s[i] = new_char; count++; }
    }
    return count;
}

```

Ersetzt alle Vorkommen von `old_char` durch `new_char` in-place und gibt die Anzahl der Ersetzungen zurück.

2.5 str_trim

```

char *str_trim(const char *s)
{
    if (s == NULL) return NULL;
    char *copy = malloc(strlen(s) + 1);
    if (!copy) return NULL;
    strcpy(copy, s);
    int start = 0;
    while (copy[start] == ' ' || copy[start] == '\t' ||
           copy[start] == '\n' || copy[start] == '\r')
        start++;
    if (start > 0)
        memmove(copy, copy + start, strlen(copy) - start + 1);
    if (copy[0] == '\0') return copy;
    int end = strlen(copy) - 1;
    while (end >= 0 && (copy[end] == ' ' || copy[end] == '\t' ||
                       copy[end] == '\n' || copy[end] == '\r'))
        end--;
    copy[end + 1] = '\0';
    return copy;
}

```

Entfernt führende und nachfolgende Whitespace-Zeichen aus einer Kopie des Strings. Der Aufrufer muss den zurückgegebenen Zeiger freigeben.

2.6 str_to_int

```

int str_to_int(const char *s, int *out)
{
    if (!s || !out) return -1;
    while (*s == ' ' || *s == '\t' || *s == '\n' || *s == '\r') s++;
    if (*s == '\0') return -1;
    int sign = 1;
    if (*s == '-') { sign = -1; s++; }
    else if (*s == '+') s++;
    if (*s < '0' || *s > '9') return -1;
    int result = 0;
    while (*s != '\0') {
        if (*s < '0' || *s > '9') return -1;
        result = result * 10 + (*s - '0');
        s++;
    }
    *out = result * sign;
    return 0;
}

```

Wandelt einen String in einen Integer um (ähnlich `atoi`, aber mit Fehlerrückgabe). Gibt -1 bei ungültigem Input, 0 bei Erfolg.

2.7 str_starts_with

```

int str_starts_with(const char *s, const char *prefix)
{
    if (s == NULL || prefix == NULL) return 0;
    int i = 0;

```



```

while (prefix[i] != '\0') {
    if (s[i] != prefix[i]) return 0;
    i++;
}
return 1;
}

```

Prüft ob s mit prefix beginnt. Gibt 1 zurueck wenn ja, 0 wenn nein oder bei NULL-Eingabe.

2.8 str_count_vowels

```

int str_count_vowels(const char *s)
{
    if (s == NULL) return 0;
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i]=='a' || s[i]=='e' || s[i]=='i' || s[i]=='o' || s[i]=='u' ||
            s[i]=='A' || s[i]=='E' || s[i]=='I' || s[i]=='O' || s[i]=='U')
            count++;
    }
    return count;
}

```

Zaehlt Vokale (groß und klein) im String durch einfachen Zeichenvergleich.

2.9 str_to_upper

```

char *str_to_upper(const char *s)
{
    if (s == NULL) return NULL;
    int len = 0;
    while (s[len] != '\0') len++;
    char *result = malloc(len + 1);
    if (result == NULL) return NULL;
    for (int i = 0; i <= len; i++) {
        if (s[i] >= 'a' && s[i] <= 'z')
            result[i] = s[i] - 32;
        else
            result[i] = s[i];
    }
    return result;
}

```

Gibt eine neu allokierte Großbuchstaben-Kopie des Strings zurück. Nutzt den ASCII-Abstand 32 zwischen Groß- und Kleinbuchstaben.

2.10 str_repeat

```

char *str_repeat(const char *s, int n)
{
    if (s == NULL || n < 0) return NULL;
    char *copy = malloc(strlen(s) * n + 1);
    if (!copy) return NULL;
    if (n == 0) { copy[0] = '\0'; return copy; }
    int count = 0;
    for (int i = 0; i < n; i++) {
        int j = 0;
        while (s[j] != '\0') { copy[count++] = s[j++]; }
    }
    copy[count] = '\0';
    return copy;
}

```

Wiederholt einen String n-mal und gibt das Ergebnis als neu allokierten String zurück.

3 Encoding / Verschlüsselung

3.1 caesar_encode

```
char *caesar_encode(const char *text, int shift)
{
    if (text == NULL) return NULL;
    size_t len = strlen(text);
    char *res = malloc(len + 1);
    if (res == NULL) return NULL;
    shift = shift % 26;
    if (shift < 0) shift += 26;
    for (size_t i = 0; i < len; i++) {
        char c = text[i];
        if (c >= 'a' && c <= 'z') res[i] = 'a' + (c - 'a' + shift) % 26;
        else if (c >= 'A' && c <= 'Z') res[i] = 'A' + (c - 'A' + shift) % 26;
        else res[i] = c;
    }
    res[len] = '\0';
    return res;
}
```

Verschlüsselt einen String mit der Caesar-Chiffre (Verschiebung um `shift` Positionen). Nicht-Buchstaben werden unverändert übernommen; der Aufrufer muss freigeben.

3.2 caesar_decode

```
char *caesar_decode(const char *text, int shift)
{
    if (text == NULL) return NULL;
    int s = shift % 26;
    if (s < 0) s += 26;
    return caesar_encode(text, (26 - s) % 26);
}
```

Entschlüsselt durch Aufruf von `caesar_encode` mit dem inversen Shift $(26-s) \% 26$. Elegante Wiederverwendung.

3.3 xor_encode

```
char *xor_encode(const char *text, uint8_t key)
{
    if (text == NULL) return NULL;
    size_t len = strlen(text);
    char *res = malloc(len + 1);
    if (res == NULL) return NULL;
    for (size_t i = 0; i < len; i++) res[i] = text[i] ^ key;
    res[len] = '\0';
    return res;
}
```

XOR-Verschlüsselung mit einem einzelnen Byte-Schlüssel. Symmetrisch: doppeltes XOR mit gleichem Key ergibt den Originaltext.

3.4 is_rotation

```
int is_rotation(const char *a, const char *b)
{
    if (a == NULL || b == NULL) return 0;
    size_t len_a = strlen(a), len_b = strlen(b);
    if (len_a != len_b) return 0;
    if (len_a == 0) return 1;
    char *concat = malloc(len_a * 2 + 1);
    if (concat == NULL) return 0;
}
```

```

    strcpy(concat, a);
    strcat(concat, a);
    int result = (strstr(concat, b) != NULL) ? 1 : 0;
    free(concat);
    return result;
}

```

Prüft ob b eine Rotation von a ist. Trick: jede Rotation von a ist ein Teilstring in aa.

3.5 rle_encode

```

char *rle_encode(const char *text)
{
    if (text == NULL) return NULL;
    size_t len = strlen(text);
    if (len == 0) {
        char *res = malloc(1);
        if (res) res[0] = '\0';
        return res;
    }
    char *res = malloc(len * 12 + 1);
    if (res == NULL) return NULL;
    int count = 1;
    char current = text[0];
    char buf[32];
    size_t pos = 0;
    for (size_t i = 1; i <= len; i++) {
        if (i < len && text[i] == current) {
            count++;
        } else {
            int w = sprintf(buf, "%d%c", count, current);
            strcpy(res + pos, buf);
            pos += w;
            if (i < len) { current = text[i]; count = 1; }
        }
    }
    char *shrink = realloc(res, pos + 1);
    return shrink ? shrink : res;
}

```

Run-Length-Encoding: "aaabbc" → "3a2b1c". Großzügige Erstallokation, dann realloc auf tatsächliche Länge.

3.6 rle_decode

```

char *rle_decode(const char *encoded)
{
    if (encoded == NULL) return NULL;
    if (encoded[0] == '\0') {
        char *res = malloc(1);
        if (res) res[0] = '\0';
        return res;
    }
    size_t total_len = 0, i = 0;
    size_t enc_len = strlen(encoded);
    while (i < enc_len) {
        if (!isdigit((unsigned char)encoded[i])) return NULL;
        int count = 0;
        while (i < enc_len && isdigit((unsigned char)encoded[i]))
            count = count * 10 + (encoded[i++] - '0');
        if (count == 0 || i >= enc_len) return NULL;
        total_len += count;
        i++;
    }
    char *res = malloc(total_len + 1);
    if (res == NULL) return NULL;
}

```

```

i = 0; size_t pos = 0;
while (i < enc_len) {
    int count = 0;
    while (i < enc_len && isdigit((unsigned char)encoded[i]))
        count = count * 10 + (encoded[i++] - '0');
    char c = encoded[i++];
    for (int k = 0; k < count; k++) res[pos++] = c;
}
res[total_len] = '\0';
return res;
}

```

Zwei-Pass-Dekodierung: erster Pass berechnet die Ausgabelänge, zweiter Pass füllt den Buffer. Gibt NULL bei ungültigem Format zurück.

3.7 compress_rle

```

void compress_rle(const char *src, char *dest)
{
    if (src[0] == '\0') { dest[0] = '\0'; return; }
    char current = src[0];
    int count = 1;
    int pos = 0;
    for (int i = 1; src[i] != '\0'; i++) {
        if (current == src[i]) count++;
        else {
            pos += sprintf(dest + pos, "%c%d", current, count);
            current = src[i];
            count = 1;
        }
    }
    sprintf(dest + pos, "%c%d", current, count);
}

```

RLE-Kompression mit Format Zeichen-dann-Zahl ("aaabbc" → "a3b2c1"). Schreibt direkt in Aufrufer-Buffer ohne Heap-Allokation – kein Größencheck!

3.8 encrypt (Caesar auf Datei)

```

void encrypt(const char *message, char offset, FILE *output)
{
    size_t len = strlen(message);
    fprintf(output, "%lu,", len);
    int shift = offset % 26;
    if (shift < 0) shift += 26;
    for (unsigned int i = 0; i < len; i++) {
        char c = message[i];
        if (c == ' ')
            fputc(' ', output);
        else {
            int pos = c - 'a';
            int encrypted = (pos + shift) % 26;
            fputc((char)('a' + encrypted), output);
        }
    }
}

```

Schreibt die verschlüsselte Nachricht in eine Datei. Das Format ist Länge;verschlüsselter_text, damit decrypt die Länge kennt.

3.9 decrypt (Caesar aus Datei)

```

char *decrypt(char offset, FILE *input)
{

```

```

if (input == NULL) return NULL;
size_t length;
if (fscanf(input, "%lu;", &length) != 1) return NULL;
char *message = malloc(length + 1);
if (message == NULL) return NULL;
int shift = offset % 26;
if (shift < 0) shift += 26;
for (size_t i = 0; i < length; i++) {
    int c = fgetc(input);
    if (c == EOF) { free(message); return NULL; }
    if (c == ' ') message[i] = ' ';
    else {
        int pos = c - 'a';
        int decrypted = pos - shift;
        if (decrypted < 0) decrypted += 26;
        message[i] = (char)('a' + decrypted);
    }
}
message[length] = '\0';
return message;
}

```

Liest Länge aus Header, allokiert Buffer und entschlüsselt zeichenweise aus der Datei. Der Aufrufer muss freigeben.

3.10 encode (RLE auf Datei)

```

void encode(const char *input, FILE *output)
{
    for (int i = 0; input[i] != '\0'; i++) {
        int count = 1;
        while (input[i] == input[i + 1]) { count++; i++; }
        fprintf(output, "%c:%d;", input[i], count);
    }
}

```

Schreibt RLE-kodierte Daten im Format Z:N; in eine Datei (z.B. a:3;b:2;c:1;).

3.11 decode (RLE aus Datei)

```

void decode(FILE *input, char *output)
{
    char c;
    int count;
    int index = 0;
    while (fscanf(input, "%c:%d;", &c, &count) == 2) {
        for (int i = 0; i < count; i++)
            output[index++] = c;
    }
    output[index] = '\0';
}

```

Liest RLE-kodierte Daten aus einer Datei und baut den Originalstring im Ausgabe-Buffer auf.

4 Rekursion

4.1 fast_power

```
long long fast_power(long long x, int n)
{
    if (n == 0) return 1;
    long long half = fast_power(x, n / 2);
    if (n % 2 == 0) return half * half;
    else return x * half * half;
}
```

Schnelle Potenzierung durch wiederholtes Quadrieren (Exponentiation by Squaring). Reduziert die Komplexität von $O(n)$ auf $O(\log n)$.

4.2 gcd

```
int gcd(int a, int b)
{
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

Euklidischer Algorithmus: $\text{ggT}(a,b) = \text{ggT}(b, a \bmod b)$, Abbruch wenn $b = 0$.

4.3 hanoi

```
int hanoi(int n, char from, char to, char aux)
{
    if (n == 0) return 0;
    int moves = 0;
    moves += hanoi(n - 1, from, aux, to);
    printf("Bewege Scheibe %d von %c nach %c\n", n, from, to);
    moves++;
    moves += hanoi(n - 1, aux, to, from);
    return moves;
}
```

Türme von Hanoi: $n - 1$ Scheiben nach aux, Scheibe n direkt, $n - 1$ Scheiben nach to. Gesamt: $2^n - 1$ Züge.

4.4 count_paths

```
int count_paths(int rows, int cols)
{
    if (rows == 1 || cols == 1) return 1;
    return count_paths(rows - 1, cols) + count_paths(rows, cols - 1);
}
```

Zählt Pfade in einem Gitter (nur rechts/runter) rekursiv. Basisfall: 1-zeilige oder 1-spaltige Gitter haben genau einen Pfad.

4.5 count_paths_memo

```
int count_paths_memo(int rows, int cols, int memo[][20])
{
    if (rows == 1 || cols == 1) return 1;
    if (memo != NULL && memo[rows - 1][cols - 1] != -1)
        return memo[rows - 1][cols - 1];
    int result = count_paths_memo(rows - 1, cols, memo)
        + count_paths_memo(rows, cols - 1, memo);
    if (memo != NULL) memo[rows - 1][cols - 1] = result;
    return result;
}
```

Wie `count_paths`, aber mit Memoization-Array (mit -1 initialisieren). Reduziert die Komplexität von exponentiell auf $O(\text{rows} \times \text{cols})$.

4.6 fib (rekursiv)

```
long long fib(int n)
{
    if (n < 0) return -1;
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Berechnet die n -te Fibonacci-Zahl rekursiv. Achtung: exponentiell langsam ohne Memoization ($O(2^n)$).

4.7 fibonacci (iterativ-rekursiv)

```
unsigned int fibonacci(unsigned int n)
{
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 2) + fibonacci(n - 1);
}
```

Alternative einfache rekursive Implementierung ohne negativen Basisfall. Ebenfalls $O(2^n)$ ohne Optimierung.

4.8 digit_sum

```
int digit_sum(int n)
{
    if (n <= 0) return 0;
    return (n % 10) + digit_sum(n / 10);
}
```

Berechnet rekursiv die Quersumme einer positiven ganzen Zahl. Extrahiert die letzte Stelle mit `% 10` und kürzt mit `/ 10`.

4.9 binary_search (rekursiv)

```
int binary_search(const int *arr, int low, int high, int target)
{
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] > target) return binary_search(arr, low, mid - 1, target);
    return binary_search(arr, mid + 1, high, target);
}
```

Rekursive Binärsuche in einem sortierten Array, $O(\log n)$. Gibt Index des Zielwerts oder -1 zurück.

4.10 count_char

```
int count_char(const char *s, char c)
{
    if (s == NULL || *s == '\0') return 0;
    if (*s == c) return 1 + count_char(s + 1, c);
    return count_char(s + 1, c);
}
```

Zählt rekursiv die Vorkommen von Zeichen `c` im String. Klassisches Beispiel für Schwanzrekursion auf Strings.

4.11 calculate_arrays

```
int calculate_arrays(const int *numbers, unsigned int length,
                    const char *operations)
{
    int result = numbers[0];
    for (unsigned int i = 0; i < length - 1; i++) {
        int next = numbers[i + 1];
        char op = operations[i];
        switch (op) {
            case '+': result = result + next; break;
            case '-': result = result - next; break;
            case '*': result = result * next; break;
            case '/': if (next != 0) result = result / next; break;
        }
    }
    return result;
}
```

Wendet eine Folge von Rechenoperationen auf ein Zahlen-Array an. Division durch Null wird abgefangen; das Ergebnis bleibt dann unverändert.

5 Structs / Strukturen

5.1 typedef

```
typedef struct {
    int    id;
    char   name[MAX_NAME_LEN];
    double grades[MAX_GRADES];
    int    grade_count;
} Student;
```

5.2 student_average

```
double student_average(const Student *s)
{
    if (s == NULL || s->grade_count == 0) return 0.0;
    double sum = s->grades[0];
    for (int i = 1; i < s->grade_count; i++) sum += s->grades[i];
    return sum / (double)s->grade_count;
}
```

Berechnet den Notendurchschnitt eines Studenten. Rückgabe 0.0 bei NULL oder keinen Noten.

5.3 find_best_student

```
Student *find_best_student(Student *students, int n)
{
    if (n <= 0 || students == NULL) return NULL;
    int best_index = 0;
    float best = student_average(&students[0]);
    for (unsigned int i = 1; i < n; i++) {
        if (student_average(&students[i]) < best) {
            best = student_average(&students[i]);
            best_index = i;
        }
    }
    return &students[best_index];
}
```

Findet den Studenten mit dem niedrigsten Notendurchschnitt (beste Note). Gibt Zeiger ins Array zurück (kein Kopieren).

5.4 sort_students_by_average

```
void sort_students_by_average(Student *students, int n)
{
    if (n <= 0 || students == NULL) return;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (student_average(&students[j]) >
                student_average(&students[j + 1])) {
                Student temp = students[j];
                students[j] = students[j + 1];
                students[j + 1] = temp;
            }
        }
    }
}
```

Bubble Sort des Studenten-Arrays nach Notendurchschnitt aufsteigend. Sortiert in-place, $O(n^2)$.

5.5 student_passed_all

```
int student_passed_all(const Student *s)
{
    if (s == NULL || s->grade_count == 0) return 0;
    for (int i = 0; i < s->grade_count; i++) {
        if (s->grades[i] > 4.0) return 0;
    }
    return 1;
}
```

Prüft ob ein Student alle Fächer bestanden hat (Note ≤ 4.0). Gibt 0 zurück sobald eine nicht bestandene Note gefunden wird.

5.6 typedef

```
typedef enum {
    ACCOUNT_CHECKING = 0,
    ACCOUNT_SAVINGS = 1
} AccountType;

typedef struct {
    int id;
    char owner[MAX_OWNER_LEN];
    AccountType type;
    double balance;
} Account;
```

5.7 account_deposit

```
int account_deposit(Account *a, double amount)
{
    if (a == NULL || amount < 0) return -1;
    a->balance += amount;
    return 0;
}
```

Buchung einer Einzahlung auf ein Konto. Gibt -1 bei ungültigem Konto oder negativem Betrag zurück.

5.8 account_withdraw

```
int account_withdraw(Account *a, double amount)
{
    if (a == NULL || amount < 0) return -1;
    if (a->balance - amount < 0) return -1;
    a->balance -= amount;
    return 0;
}
```

Hebt einen Betrag vom Konto ab. Schlagen Betragsprüfung oder Deckungsprüfung fehl, wird -1 zurückgegeben.

5.9 find_richest

```
Account *find_richest(Account *accounts, int n)
{
    if (n <= 0 || accounts == NULL) return NULL;
    int richest = accounts[0].balance;
    int index = 0;
    for (int i = 1; i < n; i++) {
        if (accounts[i].balance > richest) {
            richest = accounts[i].balance;
            index = i;
        }
    }
}
```

```

    }
    return &accounts[index];
}

```

Findet das Konto mit dem höchsten Kontostand durch lineare Suche. Gibt Zeiger ins Array zurück.

5.10 total_balance_by_type

```

double total_balance_by_type(const Account *accounts, int n,
                             AccountType type)
{
    if (n <= 0 || accounts == NULL) return 0.0;
    double balance = 0;
    for (int i = 0; i < n; i++) {
        if (accounts[i].type == type) balance += accounts[i].balance;
    }
    return balance;
}

```

Summiert alle Kontostände eines bestimmten Kontotyps. Ignoriert Konten anderen Typs.

5.11 account_transfer

```

int account_transfer(Account *from, Account *to, double amount)
{
    if (from == NULL || to == NULL || amount <= 0 ||
        from->balance < amount) return -1;
    from->balance -= amount;
    to->balance += amount;
    return 0;
}

```

Überweist einen Betrag atomar von einem Konto auf ein anderes. Prüft Deckung und Validität beider Konten.

5.12 typedef

```

struct server {
    // requests per second
    unsigned int req_per_sec;
    // utilization of machine (0-100)
    unsigned int utilization;
    // geographical location
    const char* location;
};

```

5.13 get_busiest_server

```

const struct server *get_busiest_server(const struct server *servers,
                                         unsigned int count)
{
    const struct server *busiest = &servers[0];
    for (unsigned int i = 1; i < count; i++) {
        if (servers[i].req_per_sec > busiest->req_per_sec)
            busiest = &servers[i];
    }
    return busiest;
}

```

Findet den Server mit den meisten Anfragen pro Sekunde durch lineare Suche. Gibt Zeiger auf den entsprechenden Eintrag zurück.

5.14 get_servers_in_location

```
struct server *get_servers_in_location(const struct server *servers,
                                      unsigned int count,
                                      const char *location)
{
    unsigned int match = 0;
    for (unsigned int i = 0; i < count; i++)
        if (strcmp(servers[i].location, location) == 0) match++;
    struct server *subset = malloc(match * sizeof(struct server));
    unsigned int index = 0;
    for (unsigned int i = 0; i < count; i++)
        if (strcmp(servers[i].location, location) == 0)
            subset[index++] = servers[i];
    return subset;
}
```

Filtret Server nach Standort in zwei Durchläufen: erst Zählen, dann Kopieren. Der Aufrufer muss das Ergebnis freigeben.

5.15 filter_servers_by_utilization

```
unsigned int filter_servers_by_utilization(struct server *servers,
                                          unsigned int count,
                                          unsigned int max_utilization)
{
    unsigned int counter = 0;
    for (unsigned int i = 0; i < count; i++) {
        if (servers[i].utilization <= max_utilization)
            servers[counter++] = servers[i];
    }
    return counter;
}
```

Filtret Server in-place: Server unter dem Auslastungslimit werden nach vorne kompaktiert. Gibt die neue Anzahl zurück.

5.16 typedef

```
typedef struct {
    int id;
    const char* name;
    const char* type;
    float price;
    unsigned int capacity;
} event;
```

5.17 find_event_by_id

```
const event *find_event_by_id(const event *events,
                              unsigned int size, int id)
{
    for (unsigned int i = 0; i < size; i++) {
        if (events[i].id == id) return &events[i];
    }
    return NULL;
}
```

Lineare Suche nach einer Veranstaltung anhand ihrer ID. Gibt NULL zurück wenn nicht gefunden.

5.18 find_most_expensive_event

```

int find_most_expensive_event(const event *events, unsigned int size)
{
    if (events == NULL) return -1;
    int id = events[0].id;
    float max_price = events[0].price;
    for (unsigned int i = 0; i < size; i++) {
        if (events[i].price > max_price) {
            max_price = events[i].price;
            id = events[i].id;
        }
    }
    return id;
}

```

Gibt die ID der teuersten Veranstaltung zurück. Gibt -1 bei NULL-Eingabe.

5.19 filter_events_by_type

```

event *filter_events_by_type(const event *events,
                             unsigned int size, const char *type)
{
    unsigned int count = 0;
    for (unsigned int i = 0; i < size; i++)
        if (strcmp(events[i].type, type) == 0) count++;
    event *filtered = malloc(sizeof(event) * count);
    unsigned int current = 0;
    for (unsigned int i = 0; i < size; i++)
        if (strcmp(events[i].type, type) == 0)
            filtered[current++] = events[i];
    return filtered;
}

```

Filtet Veranstaltungen nach Typ in zwei Durchläufen. Der Aufrufer muss den zurückgegebenen Zeiger freigeben.

5.20 typedef

```

typedef struct {
    unsigned int age;
    char color;
    char size;
} note;

```

5.21 sort_notes

```

int compare_notes(const void *p1, const void *p2) {
    const note *n1 = (const note *)p1;
    const note *n2 = (const note *)p2;
    if (n1->size != n2->size) return n2->size - n1->size;
    if (n1->color != n2->color) return n1->color - n2->color;
    if (n1->age < n2->age) return -1;
    if (n1->age > n2->age) return 1;
    return 0;
}

note *sort_notes(const note *notes, unsigned int length)
{
    note *sorted = malloc(sizeof(note) * length);
    memcpy(sorted, notes, sizeof(note) * length);
    qsort(sorted, length, sizeof(note), compare_notes);
    return sorted;
}

```

Sortiert Notizzettel nach Größe (absteigend), Farbe, Alter mit `qsort`. Gibt eine neu allokierte sortierte Kopie zurück.

6 Warehouse / Inventory

6.1 typedef

```
typedef struct {
    int    id;
    char   name[64];
    double price;
    int    quantity;
} Product;

typedef struct {
    Product *products;
    int     count;
    int     capacity;
    int     next_id;
} Warehouse;
```

6.2 warehouse_create

```
Warehouse *warehouse_create(void)
{
    Warehouse *w = malloc(sizeof(Warehouse) * 4);
    if (!w) return NULL;
    w->products = NULL;
    w->count = 0;
    w->capacity = 4;
    w->next_id = 1;
    return w;
}
```

Allokiert und initialisiert ein neues Warehouse. Startet mit 4 Einheiten Kapazität und ID-Zähler bei 1.

6.3 warehouse_destroy

```
void warehouse_destroy(Warehouse *w)
{
    if (!w) return;
    free(w->products);
    free(w);
}
```

Gibt zuerst das Produkt-Array, dann die Warehouse-Struktur selbst frei. Null-Zeiger wird sicher ignoriert.

6.4 warehouse_add_product

```
int warehouse_add_product(Warehouse *w, const char *name,
                          double price, int quantity)
{
    if (!w || !name) return -1;
    if (w->count >= w->capacity) {
        int new_cap = w->capacity == 0 ? 4 : w->capacity * 2;
        Product *tmp = realloc(w->products, new_cap * sizeof(Product));
        if (!tmp) return -1;
        w->products = tmp;
        w->capacity = new_cap;
    }
    w->products[w->count].id = w->next_id++;
    w->products[w->count].price = price;
    w->products[w->count].quantity = quantity;
    strncpy(w->products[w->count].name, name, 63);
    w->products[w->count].name[63] = '\0';
    w->count++;
    return 0;
}
```

```
}
```

Fügt ein Produkt hinzu und verdoppelt bei Bedarf die Kapazität mit `realloc`. `strncpy` mit manuellem Null-Terminator verhindert Buffer-Overflow.

6.5 warehouse_remove_by_id

```
int warehouse_remove_by_id(Warehouse *w, int id)
{
    if (!w) return -1;
    for (int i = 0; i < w->count; i++) {
        if (w->products[i].id == id) {
            for (int j = i; j < w->count - 1; j++)
                w->products[j] = w->products[j + 1];
            w->count--;
            return 0;
        }
    }
    return -1;
}
```

Sucht linear nach ID, löscht dann durch Linksverschiebung aller nachfolgenden Elemente.

6.6 warehouse_find_by_name

```
Product *warehouse_find_by_name(Warehouse *w, const char *query)
{
    if (!w || !query) return NULL;
    for (int i = 0; i < w->count; i++)
        if (strcmp(w->products[i].name, query) == 0)
            return &w->products[i];
    return NULL;
}
```

Lineare Suche nach Produktname. Gibt Zeiger ins Array zurück (kein Kopieren); NULL wenn nicht gefunden.

6.7 warehouse_total_value

```
double warehouse_total_value(const Warehouse *w)
{
    if (!w) return 0.0;
    double total = 0;
    for (int i = 0; i < w->count; i++)
        total += w->products[i].price * w->products[i].quantity;
    return total;
}
```

Berechnet den Gesamtwert aller Produkte als Summe von Preis $\times$ Menge.

6.8 warehouse_sort_by_price

```
void warehouse_sort_by_price(Warehouse *w)
{
    if (!w || w->count <= 1) return;
    for (int i = 0; i < w->count - 1; i++) {
        for (int j = 0; j < w->count - i - 1; j++) {
            if (w->products[j].price > w->products[j+1].price ||
                (w->products[j].price == w->products[j+1].price &&
                 w->products[j].id > w->products[j+1].id)) {
                Product tmp = w->products[j];
                w->products[j] = w->products[j+1];
                w->products[j+1] = tmp;
            }
        }
    }
}
```

```
}  
}
```

Bubble Sort nach Preis aufsteigend. Bei Preisgleichheit wird nach ID sortiert (deterministisches Ergebnis).

6.9 typedef

```
typedef struct {  
    int id;  
    char name[50];  
    char category[20];  
    float price;  
    int quantity;  
} Item;
```

6.10 find_lowest_stock

```
Item *find_lowest_stock(Item *items, int length)  
{  
    Item *lowest = &items[0];  
    for (int i = 1; i < length; i++)  
        if (items[i].quantity < lowest->quantity) lowest = &items[i];  
    return lowest;  
}
```

Findet das Item mit dem niedrigsten Lagerbestand durch lineare Suche. Gibt Zeiger ins Array zurück.

6.11 apply_discount

```
void apply_discount(Item *items, int length,  
                   const char *target_category, float discount_percent)  
{  
    for (int i = 0; i < length; i++) {  
        if (strcmp(items[i].category, target_category) == 0)  
            items[i].price = items[i].price * (1 - discount_percent / 100.0f);  
    }  
}
```

Wendet einen prozentualen Rabatt in-place auf alle Items einer Kategorie an.

7 Matrix

7.1 matrix_create / create_matrix

```
int **matrix_create(int rows, int cols)
{
    int **matrix = malloc(sizeof(int *) * rows);
    if (!matrix) return NULL;
    for (int i = 0; i < rows; i++) {
        matrix[i] = calloc(cols, sizeof(int));
        if (matrix[i] == NULL) {
            for (int j = 0; j < i; j++) free(matrix[j]);
            free(matrix);
            return NULL;
        }
    }
    return matrix;
}
```

Allokiert eine 2D-Matrix als Array von Zeigern. Bei Allokationsfehler werden bereits allokierte Zeilen sauber freigegeben (Cleanup-Loop).

7.2 matrix_free / free_matrix

```
void matrix_free(int **m, int rows)
{
    if (m == NULL) return;
    for (int i = 0; i < rows; i++) free(m[i]);
    free(m);
}
```

Gibt eine 2D-Matrix korrekt frei: erst alle Zeilen, dann das Zeigerarray.

7.3 matrix_scale

```
void matrix_scale(int **m, int rows, int cols, int factor)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            m[i][j] *= factor;
}
```

Multipliziert jedes Element der Matrix in-place mit factor.

7.4 matrix_add

```
int **matrix_add(int **a, int **b, int rows, int cols)
{
    int **matrix = matrix_create(rows, cols);
    if (!matrix) return NULL;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            matrix[i][j] = a[i][j] + b[i][j];
    return matrix;
}
```

Addiert zwei gleichgroße Matrizen elementweise und gibt das Ergebnis als neue Matrix zurück.

7.5 matrix_is_symmetric

```
int matrix_is_symmetric(int **m, int n)
{
    if (m == NULL || n <= 0) return 0;
}
```

```

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (m[i][j] != m[j][i]) return 0;
    return 1;
}

```

Prüft ob eine quadratische Matrix symmetrisch ist ($m[i][j] = m[j][i]$). Nur obere Dreiecksmatrix wird verglichen.

7.6 matrix_trace

```

int matrix_trace(int **m, int n)
{
    if (m == NULL || n <= 0) return 0;
    int sum = m[0][0];
    for (int i = 1; i < n; i++) sum += m[i][i];
    return sum;
}

```

Berechnet die Spur (Trace) einer quadratischen Matrix: Summe der Hauptdiagonalelemente.

7.7 transpose_matrix

```

int **transpose_matrix(int **matrix, int rows, int cols)
{
    int **t = create_matrix(cols, rows);
    if (!t) return NULL;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            t[j][i] = matrix[i][j];
    return t;
}

```

Transponiert eine Matrix: Element $[i][j]$ wird zu $[j][i]$. Gibt eine neu allokierte transponierte Matrix zurück.

8 Statistik

8.1 min_cycles

```
float min_cycles(const float *measurements, unsigned int count)
{
    if (measurements == NULL || count == 0) return 0;
    float temp = measurements[0];
    for (unsigned int i = 1; i < count; i++)
        if (measurements[i] < temp) temp = measurements[i];
    return temp;
}
```

Sucht das Minimum eines Float-Arrays durch lineares Durchlaufen.

8.2 max_cycles

```
float max_cycles(const float *measurements, unsigned int count)
{
    if (measurements == NULL || count == 0) return 0;
    float temp = measurements[0];
    for (unsigned int i = 1; i < count; i++)
        if (measurements[i] > temp) temp = measurements[i];
    return temp;
}
```

Sucht das Maximum eines Float-Arrays durch lineares Durchlaufen.

8.3 average_cycles

```
float average_cycles(const float *measurements, unsigned int count)
{
    if (measurements == NULL || count == 0) return 0;
    float sum = measurements[0];
    for (unsigned int i = 1; i < count; i++) sum += measurements[i];
    return sum / (float)count;
}
```

Berechnet den Mittelwert eines Float-Arrays.

8.4 standard_deviation

```
float standard_deviation(const float *measurements, unsigned int count)
{
    if (measurements == NULL || count == 0) return 0;
    float avg = average_cycles(measurements, count);
    float s = 0;
    for (unsigned int i = 0; i < count; i++)
        s += powf(measurements[i] - avg, 2);
    float variance = s / (float)count;
    return sqrtf(variance);
}
```

Berechnet die Standardabweichung: Varianz ist der mittlere quadratische Abstand vom Mittelwert, Standardabweichung ist deren Wurzel.

8.5 convert_to_time

```
float *convert_to_time(const float *measurements, unsigned int count,
                      float cycle_time)
{
    if (count == 0 || measurements == NULL) return NULL;
    float *arr = malloc(count * sizeof(float));
}
```

```
if (!arr) return NULL;
for (unsigned int i = 0; i < count; i++)
    arr[i] = measurements[i] * cycle_time;
return arr;
}
```

Multipliziert jede Messung mit der Zykluszeit um von Taktzyklen in Zeiteinheiten umzurechnen. Der Aufrufer muss freigeben.

9 Profiler

9.1 typedef

```
typedef struct {
    // name of the function
    const char* func_name;
    // how many times it was called
    unsigned int call_count;
    // how much time was spent execution all of the calls combined
    unsigned int total_cpu_time;
} function_entry;

typedef struct {
    // array containing an entry for each unique func_name (no duplicates)
    function_entry* entries;
    // number of elements currently stored in entries
    unsigned int count;
    // number of elements the entries array can hold
    unsigned int capacity;
} function_profiler;
```

9.2 create_profiler

```
function_profiler create_profiler(unsigned int capacity)
{
    function_profiler ret = {0};
    ret.entries = malloc(capacity * sizeof(function_entry));
    if (ret.entries == NULL) return ret;
    ret.capacity = capacity;
    ret.count = 0;
    return ret;
}
```

Erstellt einen Profiler mit vorgegebener Kapazität. Bei Allokationsfehler wird ein leerer (aber sicherer) Profiler zurückgegeben.

9.3 add_function_call

```
void add_function_call(function_profiler *profiler,
                      const char *func_name, unsigned int cpu_time)
{
    for (unsigned int i = 0; i < profiler->count; i++) {
        if (strcmp(profiler->entries[i].func_name, func_name) == 0) {
            profiler->entries[i].call_count++;
            profiler->entries[i].total_cpu_time += cpu_time;
            return;
        }
    }
    function_entry *entry = &profiler->entries[profiler->count];
    entry->func_name = func_name;
    entry->call_count = 1;
    entry->total_cpu_time = cpu_time;
    profiler->count++;
}
```

Registriert einen Funktionsaufruf: bei bekanntem Namen werden Aufrufzähler und CPU-Zeit akkumuliert, bei neuem Namen wird ein Eintrag angelegt.

9.4 sort_entries

```
static int compare_entries(const void *a, const void *b)
{
    const function_entry *ea = (const function_entry *)a;
```

```

    const function_entry *eb = (const function_entry *)b;
    if (ea->total_cpu_time < eb->total_cpu_time) return 1;
    if (ea->total_cpu_time > eb->total_cpu_time) return -1;
    return 0;
}

void sort_entries(function_profiler *profiler)
{
    qsort(profiler->entries, profiler->count,
          sizeof(function_entry), compare_entries);
}

```

Sortiert die Profiler-Einträge absteigend nach gesamter CPU-Zeit mit `qsort`.

9.5 destroy_profiler

```

void destroy_profiler(function_profiler *profiler)
{
    free(profiler->entries);
    profiler->entries = NULL;
    profiler->capacity = 0;
    profiler->count = 0;
}

```

Gibt den Eintrags-Puffer frei und setzt alle Felder auf null (defensiv gegen Use-after-free).

10 Einfach verkettete Liste (Singly Linked List)

10.1 typedef

```
/*
 * Knoten der verketteten Liste.
 *
 * Felder:
 *   value - gespeicherter Integer-Wert
 *   next  - Zeiger auf nachfolgenden Knoten (NULL wenn letzter)
 *
 * Hinweis: Selbstreferenz erfordert 'struct SNode' in der Definition.
 */
typedef struct SNode {
    int      value;
    struct SNode *next;
} SNode;

/*
 * Verwaltungsstruktur der Liste.
 *
 * Felder:
 *   head - Zeiger auf ersten Knoten (NULL wenn leer)
 *   tail - Zeiger auf letzten Knoten (NULL wenn leer)
 *          (tail ermöglicht O(1) push_back)
 *   size - Anzahl Knoten in der Liste
 */
typedef struct {
    SNode *head;
    SNode *tail;
    int    size;
} SList;
```

10.2 slist_create

```
SList *slist_create(void)
{
    SList *new_list = malloc(sizeof(SList));
    if (!new_list) return NULL;
    new_list->head = NULL;
    new_list->tail = NULL;
    new_list->size = 0;
    return new_list;
}
```

Allokiert und initialisiert eine leere einfach verkettete Liste. Tail-Pointer ermöglicht O(1) Anhängen.

10.3 slist_destroy

```
void slist_destroy(SList *list)
{
    if (list == NULL) return;
    SNode *current = list->head;
    while (current != NULL) {
        SNode *next_node = current->next;
        free(current);
        current = next_node;
    }
    free(list);
}
```

Gibt alle Knoten und dann die Listenstruktur frei. Wichtig: next-Pointer vor free sichern!

10.4 slist_push_front

```
int slist_push_front(SList *list, int value)
{
    if (list == NULL) return -1;
    SNode *new_node = malloc(sizeof(SNode));
    if (new_node == NULL) return -1;
    new_node->value = value;
    new_node->next = list->head;
    list->head = new_node;
    if (list->size == 0) list->tail = new_node;
    list->size++;
    return 0;
}
```

Fügt einen Wert am Anfang der Liste ein ($O(1)$). Bei leerer Liste wird auch der Tail-Pointer gesetzt.

10.5 slist_push_back

```
int slist_push_back(SList *list, int value)
{
    if (list == NULL) return -1;
    SNode *new_node = malloc(sizeof(SNode));
    if (new_node == NULL) return -1;
    new_node->value = value;
    new_node->next = NULL;
    if (list->size == 0) {
        list->head = new_node;
        list->tail = new_node;
    } else {
        list->tail->next = new_node;
        list->tail = new_node;
    }
    list->size++;
    return 0;
}
```

Hängt einen Wert ans Ende der Liste an ($O(1)$ dank Tail-Pointer). Bei leerer Liste werden Head und Tail gesetzt.

10.6 slist_pop_front

```
int slist_pop_front(SList *list, int *err)
{
    if (list == NULL || list->size == 0) {
        if (err != NULL) *err = -1;
        return 0;
    }
    SNode *remove = list->head;
    int return_value = remove->value;
    list->head = remove->next;
    list->size--;
    if (list->size == 0) list->tail = NULL;
    free(remove);
    if (err != NULL) *err = 0;
    return return_value;
}
```

Entfernt und gibt den ersten Wert zurück. Output-Parameter `err` signalisiert Fehler, da C keinen zweiten Rückgabewert kennt.

10.7 slist_remove_at


```

int slist_remove_at(SList *list, int index)
{
    if (list == NULL || index < 0 || index >= list->size) return -1;
    if (index == 0) {
        SNode *node_to_remove = list->head;
        list->head = node_to_remove->next;
        list->size--;
        if (list->size == 0) list->tail = NULL;
        free(node_to_remove);
        return 0;
    }
    SNode *prev = list->head;
    for (int i = 0; i < index - 1; i++) prev = prev->next;
    SNode *node_to_remove = prev->next;
    prev->next = node_to_remove->next;
    if (node_to_remove == list->tail) list->tail = prev;
    list->size--;
    free(node_to_remove);
    return 0;
}

```

Löscht Knoten an Position index. Index 0 ist Sonderfall (kein Vorgänger); Tail wird aktualisiert wenn letztes Element gelöscht wird.

10.8 slist_insertion_sort

```

void slist_insertion_sort(SList *list)
{
    if (list == NULL || list->size <= 1) return;
    SNode *sorted_head = NULL, *sorted_tail = NULL;
    SNode *current = list->head;
    while (current != NULL) {
        SNode *next_node = current->next;
        if (sorted_head == NULL || current->value <= sorted_head->value) {
            current->next = sorted_head;
            sorted_head = current;
            if (sorted_tail == NULL) sorted_tail = current;
        } else {
            SNode *search = sorted_head;
            while (search->next != NULL &&
                search->next->value < current->value)
                search = search->next;
            current->next = search->next;
            search->next = current;
            if (current->next == NULL) sorted_tail = current;
        }
        current = next_node;
    }
    list->head = sorted_head;
    list->tail = sorted_tail;
}

```

Sortiert die Liste in-place ohne neue Speicherzuteilung; Knoten werden direkt umgehängt. $O(n^2)$ im Worst Case.

10.9 slist_reverse

```

void slist_reverse(SList *list)
{
    if (list == NULL || list->size <= 1) return;
    SNode *prev = NULL;
    SNode *current = list->head;
    SNode *next = NULL;
    list->tail = list->head;

```

```

    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    list->head = prev;
}

```

Kehrt die Liste in-place mit drei Zeigern um ($O(n)$, $O(1)$ Speicher). Alter Head wird zum neuen Tail.

10.10 slist_to_array

```

int *slist_to_array(const SList *list)
{
    if (list == NULL || list->size == 0) return NULL;
    int *array = malloc(list->size * sizeof(int));
    if (array == NULL) return NULL;
    SNode *current = list->head;
    for (int i = 0; i < list->size; i++) {
        array[i] = current->value;
        current = current->next;
    }
    return array;
}

```

Kopiert alle Listenelemente in ein neu allokiertes Array. Der Aufrufer muss das Array mit `free()` freigeben.

10.11 list_append (einfache Variante)

```

void list_append(Node **head, int value)
{
    if (head == NULL) return;
    Node *new_node = malloc(sizeof(Node));
    if (new_node == NULL) return;
    new_node->data = value;
    new_node->next = NULL;
    if (*head == NULL) { *head = new_node; return; }
    Node *curr = *head;
    while (curr->next != NULL) curr = curr->next;
    curr->next = new_node;
}

```

Hängt einen Wert ans Ende einer einfachen Liste an (kein Tail-Pointer, daher $O(n)$).

10.12 list_free

```

void list_free(Node **head)
{
    if (head == NULL) return;
    Node *curr = *head;
    while (curr != NULL) {
        Node *next = curr->next;
        free(curr);
        curr = next;
    }
    *head = NULL;
}

```

Gibt alle Knoten frei und setzt den Head-Zeiger auf NULL. Doppelzeiger verhindert Dangling Pointer.

10.13 list_sum

```
int list_sum(const Node *head)
{
    int sum = 0;
    const Node *curr = head;
    while (curr != NULL) { sum += curr->data; curr = curr->next; }
    return sum;
}
```

Summiert alle Werte der Liste in einem einzigen Durchlauf.

10.14 list_remove_duplicates

```
void list_remove_duplicates(Node **head)
{
    if (head == NULL || *head == NULL) return;
    Node *curr = *head;
    while (curr != NULL) {
        Node *runner = curr;
        while (runner->next != NULL) {
            if (runner->next->data == curr->data) {
                Node *duplicate = runner->next;
                runner->next = runner->next->next;
                free(duplicate);
            } else {
                runner = runner->next;
            }
        }
        curr = curr->next;
    }
}
```

Entfernt alle Duplikate mit einem äußeren und einem inneren Zeiger (Runner-Technik). $O(n^2)$, benötigt keinen zusätzlichen Speicher.

10.15 list_is_sorted

```
int list_is_sorted(const Node *head)
{
    if (head == NULL || head->next == NULL) return 1;
    const Node *curr = head;
    while (curr->next != NULL) {
        if (curr->data > curr->next->data) return 0;
        curr = curr->next;
    }
    return 1;
}
```

Prüft ob die Liste aufsteigend sortiert ist. Gibt sofort 0 zurück beim ersten Verstoß.

10.16 list_insert_sorted

```
void list_insert_sorted(Node **head, int value)
{
    if (head == NULL) return;
    Node *new_node = malloc(sizeof(Node));
    if (new_node == NULL) return;
    new_node->data = value;
    if (*head == NULL || (*head)->data >= value) {
        new_node->next = *head;
        *head = new_node;
        return;
    }
    Node *curr = *head;
    while (curr->next != NULL && curr->next->data < value)
```

```
    curr = curr->next;
    new_node->next = curr->next;
    curr->next = new_node;
}
```

Fügt einen Wert an der richtigen Position in eine bereits sortierte Liste ein. Einfügen am Kopf ist Sonderfall.

10.17 list_filter_even

```
void list_filter_even(Node **head_ref)
{
    Node *current = *head_ref;
    while (current != NULL && current->next != NULL) {
        if (current->next->data % 2 == 0) {
            Node *del = current->next;
            current->next = del->next;
            free(del);
        } else {
            current = current->next;
        }
    }
}
```

Entfernt alle geraden Werte aus der Liste (über den Nachfolger). Achtung: der Head wird nicht geprüft!

11 Doppelt verkettete Liste (Doubly Linked List)

11.1 typedef

```
typedef struct DNode {
    int      data;
    struct DNode *next;
    struct DNode *prev;
} DNode;

typedef struct {
    DNode *head;
    DNode *tail;
    int    size;
} DList;
```

11.2 dlist_create

```
DList *dlist_create(void)
{
    DList *l = malloc(sizeof(DList));
    if (!l) return NULL;
    l->head = NULL;
    l->tail = NULL;
    l->size = 0;
    return l;
}
```

Allokiert und initialisiert eine leere doppelt verkettete Liste mit Head- und Tail-Zeiger.

11.3 dlist_destroy

```
void dlist_destroy(DList *list)
{
    if (!list) return;
    DNode *cur = list->head;
    while (cur) {
        DNode *nxt = cur->next;
        free(cur);
        cur = nxt;
    }
    free(list);
}
```

Gibt alle Knoten von vorne nach hinten und anschließend die Listenstruktur frei.

11.4 dlist_push_front

```
int dlist_push_front(DList *list, int value)
{
    if (list == NULL) return -1;
    DNode *new_node = malloc(sizeof(DNode));
    if (new_node == NULL) return -1;
    new_node->data = value;
    new_node->next = list->head;
    new_node->prev = NULL;
    if (list->head != NULL) list->head->prev = new_node;
    list->head = new_node;
    if (list->size == 0) list->tail = new_node;
    list->size++;
    return 0;
}
```

Fügt am Kopf ein und setzt alle vier Zeigerketten korrekt (prev/next des neuen Knotens und prev des alten Heads).

11.5 dlist_push_back

```
int dlist_push_back(DList *list, int value)
{
    if (list == NULL) return -1;
    DNode *new_node = malloc(sizeof(DNode));
    if (new_node == NULL) return -1;
    new_node->data = value;
    new_node->next = NULL;
    new_node->prev = NULL;
    if (list->size == 0) {
        list->head = new_node;
        list->tail = new_node;
    } else {
        new_node->prev = list->tail;
        list->tail->next = new_node;
        list->tail = new_node;
    }
    list->size++;
    return 0;
}
```

Hängt am Ende an und aktualisiert Tail-Zeiger sowie den Prev-Zeiger des neuen Knotens.

11.6 dlist_pop_back

```
int dlist_pop_back(DList *list, int *err)
{
    if (list == NULL || list->size == 0 || list->tail == NULL) {
        if (err != NULL) *err = -1;
        return 0;
    }
    if (err != NULL) *err = 0;
    DNode *to_remove = list->tail;
    int value = to_remove->data;
    list->tail = to_remove->prev;
    if (list->tail != NULL) list->tail->next = NULL;
    else list->head = NULL;
    list->size--;
    free(to_remove);
    return value;
}
```

Entfernt den letzten Knoten in $O(1)$ dank Tail-Zeiger. Wenn die Liste dadurch leer wird, muss auch Head auf NULL gesetzt werden.

11.7 dlist_is_consistent

```
int dlist_is_consistent(const DList *list)
{
    if (list == NULL) return 1;
    if (list->size == 0)
        return (list->head == NULL && list->tail == NULL) ? 1 : 0;
    if (list->head == NULL || list->tail == NULL) return 0;
    if (list->head->prev != NULL) return 0;
    if (list->tail->next != NULL) return 0;
    const DNode *curr = list->head;
    int count = 0;
    while (curr != NULL) {
        count++;
        if (curr->next != NULL && curr->next->prev != curr) return 0;
        if (curr->next == NULL && curr != list->tail) return 0;
        curr = curr->next;
    }
    return (count == list->size) ? 1 : 0;
}
```

```
}
```

Validiert alle Invarianten der doppelt verketteten Liste (Zeigerkonsistenz, Größe, prev/next-Verkettung). Nützlich für Debugging.

12 Stack

12.1 typedef

```
typedef struct {
    int *data;      /* dynamisches Array */
    int size;       /* aktuelle Anzahl Elemente */
    int capacity;   /* aktuell allokierte Kapazität */
} Stack;
```

12.2 stack_create

```
Stack *stack_create(void)
{
    Stack *s = malloc(sizeof(Stack));
    if (s == NULL) return NULL;
    s->data = malloc(4 * sizeof(int));
    if (s->data == NULL) { free(s); return NULL; }
    s->size = 0;
    s->capacity = 4;
    return s;
}
```

Erstellt einen dynamischen Stack (als Array) mit initialer Kapazität 4. Bei Allokationsfehler des Datenarrays wird s sauber freigegeben.

12.3 stack_destroy

```
void stack_destroy(Stack *s)
{
    if (s == NULL) return;
    if (s->data != NULL) free(s->data);
    free(s);
}
```

Gibt Daten-Array und Stack-Struktur frei. Null-Zeiger wird sicher ignoriert.

12.4 stack_push

```
int stack_push(Stack *s, int value)
{
    if (s == NULL) return -1;
    if (s->size == s->capacity) {
        int new_capacity = (s->capacity == 0) ? 4 : s->capacity * 2;
        int *new_data = realloc(s->data, new_capacity * sizeof(int));
        if (new_data == NULL) return -1;
        s->data = new_data;
        s->capacity = new_capacity;
    }
    s->data[s->size] = value;
    s->size++;
    return 0;
}
```

Schiebt einen Wert auf den Stack. Bei voller Kapazität wird mit realloc auf doppelte Größe vergrößert.

12.5 stack_pop

```
int stack_pop(Stack *s, int *err)
{
    if (s == NULL || s->size == 0) {
        if (err != NULL) *err = -1;
        return 0;
    }
}
```



```

    }
    if (err != NULL) *err = 0;
    s->size--;
    return s->data[s->size];
}

```

Entnimmt das oberste Element (LIFO). Output-Parameter `err` signalisiert Underflow.

12.6 stack_peek

```

int stack_peek(const Stack *s, int *err)
{
    if (s == NULL || s->size == 0) {
        if (err != NULL) *err = -1;
        return 0;
    }
    if (err != NULL) *err = 0;
    return s->data[s->size - 1];
}

```

Liest das oberste Element ohne es zu entfernen (kein Größenänderung).

12.7 stack_is_empty

```

int stack_is_empty(const Stack *s)
{
    if (s == NULL || s->size == 0) return 1;
    return 0;
}

```

Gibt 1 zurück wenn der Stack leer oder NULL ist, sonst 0.

13 Queue (Warteschlange)

13.1 typedef

```
typedef struct
{
    int *data; /* dynamisches Array */
    int front; /* Index des ersten Elements */
    int back; /* Index hinter dem letzten Element */
    int size; /* aktuelle Anzahl Elemente */
    int capacity; /* aktuell allokierte Kapazität */
} Queue;
```

13.2 queue_create

```
Queue *queue_create(void)
{
    Queue *new_queue = malloc(sizeof(Queue));
    if (!new_queue) return NULL;
    new_queue->data = malloc(4 * sizeof(int));
    if (new_queue->data == NULL) { free(new_queue); return NULL; }
    new_queue->back = 0;
    new_queue->front = 0;
    new_queue->size = 0;
    new_queue->capacity = 4;
    return new_queue;
}
```

Erstellt eine Queue als zyklisches Array mit initialer Kapazität 4. Front und Back zeigen auf Index 0.

13.3 queue_destroy

```
void queue_destroy(Queue *q)
{
    if (q == NULL) return;
    if (q->data != NULL) free(q->data);
    free(q);
}
```

Gibt den Datenpuffer und die Queue-Struktur frei.

13.4 queue_enqueue

```
int queue_enqueue(Queue *q, int value)
{
    if (q == NULL) return -1;
    if (q->size == q->capacity) {
        int new_capacity = (q->capacity == 0) ? 4 : q->capacity * 2;
        int *new_data = malloc(new_capacity * sizeof(int));
        if (new_data == NULL) return -1;
        for (int i = 0; i < q->size; i++)
            new_data[i] = q->data[(q->front + i) % q->capacity];
        free(q->data);
        q->data = new_data;
        q->front = 0;
        q->back = q->size;
        q->capacity = new_capacity;
    }
    q->data[q->back] = value;
    q->back = (q->back + 1) % q->capacity;
    q->size++;
    return 0;
}
```

Fügt am Ende der Queue ein. Bei Kapazitätsüberschreitung: neues Array allokalieren und Daten linearisieren (zyklische Reihenfolge auflösen).

13.5 queue_dequeue

```
int queue_dequeue(Queue *q, int *err)
{
    if (q == NULL || q->size == 0) {
        if (err != NULL) *err = -1;
        return 0;
    }
    int value = q->data[q->front];
    q->front = (q->front + 1) % q->capacity;
    q->size--;
    if (err != NULL) *err = 0;
    return value;
}
```

Entnimmt das vorderste Element (FIFO) mit zyklischer Indexarithmetik.

13.6 queue_peek

```
int queue_peek(const Queue *q, int *err)
{
    if (q == NULL || q->size == 0) {
        if (err != NULL) *err = -1;
        return 0;
    }
    if (err != NULL) *err = 0;
    return q->data[q->front];
}
```

Liest das vorderste Element ohne Entnahme.

13.7 queue_is_empty

```
int queue_is_empty(const Queue *q)
{
    if (q == NULL) return 1;
    return q->size == 0;
}
```

Gibt 1 zurück wenn die Queue leer oder NULL ist.

13.8 queue_size

```
int queue_size(const Queue *q)
{
    if (q == NULL) return 0;
    return q->size;
}
```

Gibt die aktuelle Anzahl der Elemente in der Queue zurück.

14 Dynamisches Array

14.1 typedef

```
typedef struct {  
    double *data; /* dynamisches Array */  
    int size; /* aktuelle Anzahl Elemente */  
    int capacity; /* aktuell allokierte Kapazitaet */  
} DynArray;
```

14.2 da_create

```
DynArray *da_create(void)  
{  
    DynArray *a = malloc(sizeof(DynArray));  
    if (!a) return NULL;  
    a->data = malloc(4 * sizeof(double));  
    if (!a->data) { free(a); return NULL; }  
    a->size = 0;  
    a->capacity = 4;  
    return a;  
}
```

Erstellt ein dynamisches Array für double-Werte mit Startkapazität 4.

14.3 da_destroy

```
void da_destroy(DynArray *a)  
{  
    if (a) {  
        if (a->data) free(a->data);  
        free(a);  
    }  
}
```

Gibt Datenpuffer und Array-Struktur frei.

14.4 da_push

```
int da_push(DynArray *a, double value)  
{  
    if (!a) return -1;  
    if (a->size == a->capacity) {  
        int new_capacity = (a->capacity == 0) ? 4 : a->capacity * 2;  
        double *new_data = realloc(a->data, new_capacity * sizeof(double));  
        if (!new_data) return -1;  
        a->data = new_data;  
        a->capacity = new_capacity;  
    }  
    a->data[a->size] = value;  
    a->size++;  
    return 0;  
}
```

Fügt einen Wert ans Ende an und verdoppelt bei Bedarf die Kapazität mit realloc.

14.5 da_get

```
double da_get(const DynArray *a, int index, int *err)  
{  
    if (!a || index < 0 || index >= a->size) {  
        if (err) *err = -1;  
        return 0.0;  
    }  
}
```

```

    }
    if (err) *err = 0;
    return a->data[index];
}

```

Gibt Element an Index mit Bereichsprüfung zurück. Fehler wird über Output-Parameter `err` signalisiert.

14.6 da_remove_at

```

int da_remove_at(DynArray *a, int index)
{
    if (!a || index < 0 || index >= a->size) return -1;
    for (int i = index; i < a->size - 1; i++)
        a->data[i] = a->data[i + 1];
    a->size--;
    return 0;
}

```

Löscht Element an Position durch Linksverschiebung aller Nachfolger.

14.7 da_min

```

double da_min(const DynArray *a, int *err)
{
    if (!a || a->size == 0) { if (err) *err = -1; return 0.0; }
    double min_val = a->data[0];
    for (int i = 1; i < a->size; i++)
        if (a->data[i] < min_val) min_val = a->data[i];
    if (err) *err = 0;
    return min_val;
}

```

Findet das Minimum im dynamischen Array durch lineares Durchlaufen.

14.8 da_max

```

double da_max(const DynArray *a, int *err)
{
    if (!a || a->size == 0) { if (err) *err = -1; return 0.0; }
    double max_val = a->data[0];
    for (int i = 1; i < a->size; i++)
        if (a->data[i] > max_val) max_val = a->data[i];
    if (err) *err = 0;
    return max_val;
}

```

Findet das Maximum im dynamischen Array durch lineares Durchlaufen.

14.9 da_mean

```

double da_mean(const DynArray *a, int *err)
{
    if (!a || a->size == 0) { if (err) *err = -1; return 0.0; }
    double sum = 0.0;
    for (int i = 0; i < a->size; i++) sum += a->data[i];
    if (err) *err = 0;
    return sum / a->size;
}

```

Berechnet den Mittelwert aller Elemente im dynamischen Array.

14.10 da_sort

```
static int compare_doubles(const void *p1, const void *p2)
{
    double d1 = *(const double *)p1;
    double d2 = *(const double *)p2;
    if (d1 < d2) return -1;
    if (d1 > d2) return 1;
    return 0;
}

void da_sort(DynArray *a)
{
    if (!a || a->size <= 1) return;
    qsort(a->data, a->size, sizeof(double), compare_doubles);
}
```

Sortiert das Array aufsteigend mit `qsort`. Die Vergleichsfunktion muss explizit über Zeiger arbeiten da `qsort` `void*` erwartet.

15 File I/O

15.1 count_lines

```
int count_lines(const char *filename)
{
    if (filename == NULL) return -1;
    FILE *file = fopen(filename, "r");
    if (!file) return -1;
    int count = 0;
    int ch;
    int last_char = EOF;
    while ((ch = fgetc(file)) != EOF) {
        if (ch == '\n') count++;
        last_char = ch;
    }
    if (last_char != EOF && last_char != '\n') count++;
    fclose(file);
    return count;
}
```

Zählt Zeilen in einer Datei zeichenweise. Letzte Zeile ohne abschließendes `\n` wird durch Sonderfall korrekt gezählt.

15.2 read_ints_from_file

```
int read_ints_from_file(const char *filename, int *out, int max_count)
{
    if (filename == NULL || out == NULL || max_count <= 0) return -1;
    FILE *file = fopen(filename, "r");
    if (file == NULL) return -1;
    int count = 0;
    char buffer[256];
    while (count < max_count && fgets(buffer, sizeof(buffer), file)) {
        int value;
        if (sscanf(buffer, "%d", &value) == 1) {
            out[count] = value;
            count++;
        }
    }
    fclose(file);
    return count;
}
```

Liest Integers zeilenweise aus einer Datei in ein Array. Nicht-Integer-Zeilen werden übersprungen; gibt Anzahl gelesener Werte zurück.

15.3 file_max

```
int file_max(const char *filename, int *out_max)
{
    if (filename == NULL || out_max == NULL) return -1;
    FILE *file = fopen(filename, "r");
    if (file == NULL) return -1;
    int max_val, found_any = 0;
    char buffer[256];
    while (fgets(buffer, sizeof(buffer), file)) {
        int val;
        if (sscanf(buffer, "%d", &val) == 1) {
            if (!found_any) { max_val = val; found_any = 1; }
            else if (val > max_val) max_val = val;
        }
    }
    fclose(file);
    if (!found_any) return -1;
}
```

```

    *out_max = max_val;
    return 0;
}

```

Findet den größten Integer in einer Datei. Gibt -1 wenn keine Zahlen gefunden wurden, sonst 0 und Ergebnis über out_max.

15.4 write_squares

```

int write_squares(const char *filename, int n)
{
    if (filename == NULL || n <= 0) return -1;
    FILE *file = fopen(filename, "w");
    if (file == NULL) return -1;
    for (int i = 1; i <= n; i++) fprintf(file, "%d\n", i * i);
    fclose(file);
    return 0;
}

```

Schreibt die Quadratzahlen 1 bis n^2 in eine Datei (eine pro Zeile).

15.5 number_lines

```

int number_lines(const char *src, const char *dst)
{
    if (src == NULL || dst == NULL) return -1;
    FILE *file1 = fopen(src, "r");
    if (file1 == NULL) return -1;
    FILE *file2 = fopen(dst, "w");
    if (file2 == NULL) { fclose(file1); return -1; }
    int count = 0;
    int ch, new_line = 1;
    while ((ch = fgetc(file1)) != EOF) {
        if (new_line) {
            count++;
            fprintf(file2, "%d: ", count);
            new_line = 0;
        }
        fputc(ch, file2);
        if (ch == '\n') new_line = 1;
    }
    fclose(file1);
    fclose(file2);
    return count;
}

```

Kopiert src nach dst und stellt jeder Zeile ihre Nummer voran. Gibt die Gesamtzahl der Zeilen zurück.